Updated tagging strategy for Kaizen Regtech company for its AWS estate

Define Objectives for Tagging

Kaizen's RegTech company's tagging strategy is an enhancement to the existing framework to provide resource governance, accountability, regulatory compliance, operational efficiency and cost management. The aim is to enable consistent tagging across AWS landscape of resources to simplify governance, improve traceability, and ensure compliance with regulatory standards.

The objectives include:

- **Environment**: Aligning with deployment environments (e.g., dev, test, prod).
- **Owner**: Designating the resource owner for accountability (e.g., devops, data, squad-<alphanumeric>).
- **Project**: Associating resources with specific projects for auditing and access control (e.g., Portal).
- **PortalModule**: Associating resources with regulatory compliance modules (e.g., CRS, KRDP-EMIR).
- **CostCenter**: Enabling cost allocation and tracking for departments or projects (e.g., CC-<number>).
- **DataClassification**: Classifying data sensitivity for compliance (e.g., confidential, public, restricted).

This enhanced strategy provides a foundation for effective resource management, cost optimization, and regulatory compliance across the AWS infrastructure.

Tagging Schema

The tagging schema defines the required and optional tags, their formats, and their usage. Below is the proposed schema:

List of values need to be updated

Tag Key	Purpose	Value Format	Example	Required
Environment	Identify the environment (dev, test, prod)	Fixed List (dev, test, prod)	prod	Yes
Owner	Resource owner for accountability	Free Text (user/team name)	devops data squad-<alphanumeric >	Yes
Project	Associate resources with a project	Free Text	Portal KRDP	Yes
PortalModule	Associate with the regulation	Free Text	CRS	Yes

DataClassification	Classify data sensitivity	Fixed List (confidential, public, restricted)	confidential	Optional (for data storage or DB resources)
CostCenter	Map costs to departments or cost centers	Fixed Pattern (CC-<number>)	CC-12345	Yes (but for future proofing)
Subsidiary	Identify the Subsidiary a sandbox account (AWS Account Alias) belongs to	Free Text	srb sds	yes ***
Backup	Identify backup requirements	Fixed List (yes, no)	yes	Optional (for backup resources)
RetentionClassification	Define data retention requirements	Fixed List (30d, 1y, 7y)	7y	Optional (for backup and data retention)

Tagging Guidelines

Notes:

***Account names can't be tagged. Use metadata tags (e.g., `subsidiary = srb`) via AWS Organizations. Tagging is supported at creation time in CloudFormation/Terraform. Use the AWS CLI, SDK, or Console to tag existing accounts.

- **Mandatory Tags**: Environment, Owner, Project, PortalModule, and CostCenter
- **Optional Tags**: Backup, DataClassification, and RetentionClassification

  The optional tags may be applicable for certain AWS resources like S3 Bucket, Amazon RDS, FSx for windows only.

```
Resources:
  MyS3Bucket:
    Type: AWS::S3::Bucket
    Properties:
      BucketName: example-bucket
      Tags:
        - Key: Environment
          Value: prod
        - Key: Owner
          Value: devops-team
        - Key: Project
          Value: CustomerPortal
```

```
        - Key: DataClassification
          Value: confidential
        - Key: Backup
          Value: yes
        - Key: RetentionClassification
          Value: 7y
        - Key: CostCenter
          Value: CC-12345
```

Implement Tagging

Automated Tagging:

- Use **AWS Service Catalog** and **AWS CloudFormation** templates to enforce consistent tagging during resource provisioning.
- Implement **Tag Policies** in AWS Organizations to ensure compliance with the tagging schema.

Retroactive Tagging:

- Use the **Resource Groups Tagging API** or **AWS CLI** to tag existing resources.
  - Example CLI:

```
aws ec2 create-tags --resources i-1234567890abcdef0 --tags Key=environment,Value=prod
Key=owner,Value=compliance-team
```

Prevent Tagging Drift:

- Use **AWS Config** rules to monitor and enforce tagging compliance.

Monitor and Refine the Tagging Strategy

- **Tag Utilization Reports**: Use **AWS Cost Explorer** and **AWS Budgets** to track costs by tags.
- **Continuous Improvement**: Regular reviews with stakeholders (compliance officers, engineering teams), and schema updates as business or regulatory requirements evolve.

Tagging Enforcement with SCPs and Tag Policies

Guidelines for SCP Management

1. **Use Modular Statements:** Break down policies by service or resource type for improved readability and maintainability.
2. **Descriptive SIDs:** Use meaningful `Sid` values for easy identification of the purpose of each statement.
3. **Consistency:** Ensure consistent tagging keys and values across all SCPs.
4. **Validation:** Test each SCP in a staging environment before applying them organization-wide.

SCP Example:

We will use **Service Control Policies (SCPs)** applied at the AWS Organisation Master account to enforce tagging policies across all accounts in the organization.

- Example SCP to enforce the Environment, Project, and PortalModule tags:

json

Example Modular SCP for EC2

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Sid": "EnforceEnvironmentTagsForEC2",
      "Effect": "Deny",
      "Action": "ec2:RunInstances",
      "Resource": "*",
      "Condition": {
        "StringNotEquals": {
          "aws:RequestTag/Environment": ["prod", "test", "dev"],
          "aws:RequestTag/Project": ["Portal"],
          "aws:RequestTag/PortalModule": ["KRDP-EMIR"]
        }
      }
    }
  ]
}
```

Example Modular SCP for AWS Glue

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Sid": "EnforceTagsForGlueJobs",
      "Effect": "Deny",
      "Action": [
        "glue:CreateJob",
        "glue:UpdateJob"
      ],
      "Resource": "*",
      "Condition": {
        "StringNotEquals": {
          "aws:RequestTag/Environment": ["prod", "test", "dev"],
          "aws:RequestTag/Project": ["Portal"],
          "aws:RequestTag/PortalModule": ["KRDP-EMIR"]
        }
      }
    }
  ]
}

```

```
```

Example Modular SCP for S3 Buckets

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Sid": "EnforceMandatoryTagsForS3",
      "Effect": "Deny",
      "Action": [
        "s3:CreateBucket",
        "s3:PutBucketTagging"
      ],
      "Resource": "*",
      "Condition": {
        "StringNotEquals": {
          "aws:RequestTag/Environment": ["prod", "test", "dev"],
          "aws:RequestTag/Project": ["Portal"],
          "aws:RequestTag/PortalModule": ["KRDP-EMIR"],
          "aws:RequestTag/DataClassification": ["confidential", "public",
"restricted"],
          "aws:RequestTag/RetentionClassification": ["30d", "1y", "7y"]
        }
      }
    }
  ]
}
```

Example Modular SCP for RDS Instances

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Sid": "EnforceTagsForRDSInstances",
      "Effect": "Deny",
      "Action": [
        "rds:CreateDBInstance",
        "rds:CreateDBCluster",
        "rds:AddTagsToResource"
      ],
      "Resource": "*",
      "Condition": {
        "StringNotEquals": {
          "aws:RequestTag/Environment": ["prod", "test", "dev"],
          "aws:RequestTag/Project": ["Portal"],
          "aws:RequestTag/PortalModule": ["KRDP-EMIR"],
          "aws:RequestTag/DataClassification": ["confidential", "public",
"restricted"],
          "aws:RequestTag/Backup": ["yes", "no"],
          "aws:RequestTag/RetentionClassification": ["30d", "1y", "7y"]
        }
      }
```

```
    }
  ]
}
```

Example Modular SCP for DynamoDB

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Sid": "EnforceDynamoDBTags",
      "Effect": "Deny",
      "Action": [
        "dynamodb:CreateTable",
        "dynamodb:TagResource"
      ],
      "Resource": "*",
      "Condition": {
        "StringNotEquals": {
          "aws:RequestTag/Environment": ["prod", "test", "dev"],
          "aws:RequestTag/Project": ["Portal"],
          "aws:RequestTag/PortalModule": ["KRDP-EMIR"],
          "aws:RequestTag/DataClassification": ["confidential", "public",
"restricted"],
          "aws:RequestTag/RetentionClassification": ["30d", "1y", "7y"]
        }
      }
    }
  ]
}
```

Example Modular SCP for Lambda Functions

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Sid": "EnforceLambdaTags",
      "Effect": "Deny",
      "Action": [
        "lambda:CreateFunction",
        "lambda:TagResource",
        "lambda:PublishVersion"
      ],
      "Resource": "*",
      "Condition": {
        "StringNotEquals": {
          "aws:RequestTag/Environment": ["prod", "test", "dev"],
          "aws:RequestTag/Project": ["Portal"],
          "aws:RequestTag/PortalModule": ["KRDP-EMIR"]
        }
      }
    }
  ]
```

```
}
```

Example Modular SCP for ECS Clusters

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Sid": "EnforceECSClusterTags",
      "Effect": "Deny",
      "Action": [
        "ecs:CreateCluster",
        "ecs:CreateService",
        "ecs:TagResource"
      ],
      "Resource": "*",
      "Condition": {
        "StringNotEquals": {
          "aws:RequestTag/Environment": ["prod", "test", "dev"],
          "aws:RequestTag/Project": ["Portal"],
          "aws:RequestTag/PortalModule": ["KRDP-EMIR"]
        }
      }
    }
  ]
}
```

Example Modular SCP for CloudWatch Log Groups

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Sid": "EnforceCloudWatchLogGroupTags",
      "Effect": "Deny",
      "Action": [
        "logs:CreateLogGroup",
        "logs:TagLogGroup"
      ],
      "Resource": "*",
      "Condition": {
        "StringNotEquals": {
          "aws:RequestTag/Environment": ["prod", "test", "dev"],
          "aws:RequestTag/Project": ["Portal"],
          "aws:RequestTag/PortalModule": ["KRDP-EMIR"],
          "aws:RequestTag/RetentionClassification": ["30d", "1y", "7y"]
        }
      }
    }
  ]
}

```

Tag Policies:

- **Tag Policies** in **AWS Organizations** can enforce valid tag keys and values for all accounts in the organization.
  Example:

json

```
{
  "tags": {
    "Environment": {
      "tag_key": {
        "enforced_for_create": true
      },
      "tag_value": {
        "allowed_values": ["prod", "test", "dev"]
      }
    },
    "Project": {
      "tag_key": {
        "enforced_for_create": true
      },
      "tag_value": {
        "allowed_values": ["Portal"]
      }
    },
    "PortalModule": {
      "tag_key": {
        "enforced_for_create": true
      },
      "tag_value": {
        "allowed_values": ["KRDP-EMIR"]
      }
    }
  }
}

```

Operational Efficiency and Compliance

Centralized Governance with AWS Organizations

- AWS Organizations enables the central management of multiple AWS accounts.
- Use AWS Organizations Stack Sets for multi-account tag deployment.
- Centralized scaling of tagging strategy across accounts and regions.

Automated Tagging and Compliance

- Use AWS CloudFormation and AWS Service Catalog to automatically apply tags during resource creation.
- Leverage AWS EventBridge rules for automated tag compliance checking.
- Utilize AWS Lambda functions for automated tag correction and remediation.

Tag Management and Updates

- Use the Resource Groups Tagging API or AWS CLI for retroactive tagging of existing resources.
- Employ AWS Systems Manager automation for bulk tagging updates.

- Streamlined processes for large-scale tag modifications.

Monitor and Enforce Consistency

- Use **AWS Config** to monitor resources for missing or incorrect tags. This ensures compliance with mandatory tags like Environment and Owner.
- **AWS Security Hub** and **AWS Audit Manager** can be used to track and report compliance against regulatory tagging requirements.

Cost Reporting by Tags

- AWS Cost Explorer can analyze costs by tags, helping to monitor and allocate costs efficiently across resources.
- AWS Budgets alerts can be configured to notify stakeholders of spending thresholds based on tag filters.
- Cost Allocation Tags can be activated in the AWS Billing and Cost Management Console to track spending by key tags like Environment, Client, and CostCenter.

Backup and Validate Tags

- Use AWS Resource Groups and Cost Allocation Reports to identify and audit tagged resources.
- Automated scripts can back up existing tags to S3 for recovery.